

PromptLab

Memoria del Proyecto de ALS

- **Autor:** Alejandro López Gómez
- **DNI:** 32723985E
- **Asignatura:** Aplicaciones para Lenguajes y Sistemas
- **Curso:** 2025/2026
- **Repositorio:** <https://github.com/AlexLopezGomez/ALS-PromptLab>

1. Introducción

PromptLab es una aplicación web desarrollada en Flask que permite a una comunidad de usuarios gestionar, organizar, comentar y evaluar *prompts* de modelos de inteligencia artificial. Cada usuario puede registrarse, crear prompts propios, agruparlos en colecciones temáticas, navegar por los prompts del resto de la comunidad, comentarlos y puntuarlos.

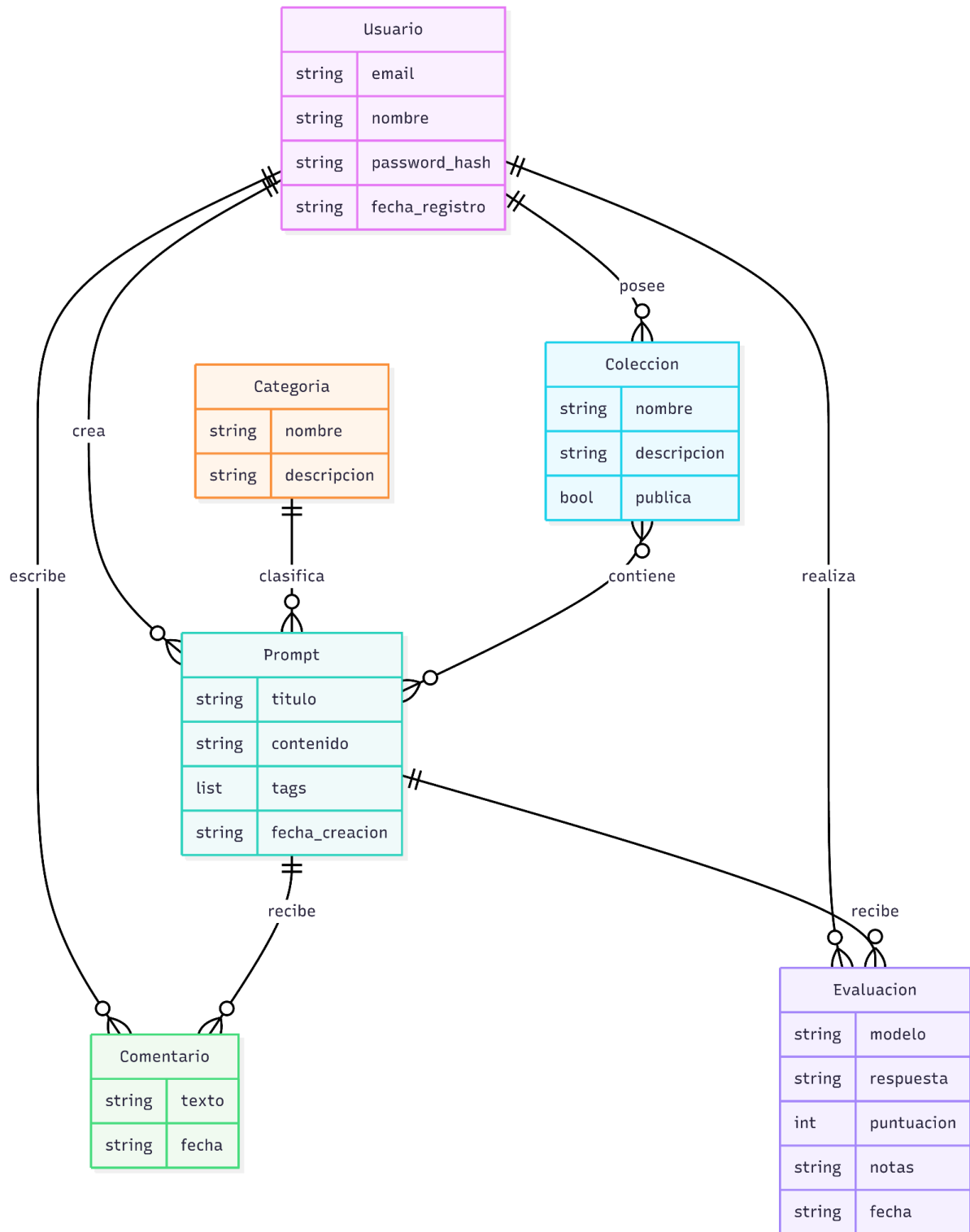
El proyecto cumple los requisitos técnicos del enunciado: aplicación web con **Flask**, plantillas **Jinja2**, autenticación con **Flask-Login** y persistencia mediante **Sirope** sobre **Redis**. Se han implementado cinco entidades de dominio además del usuario, lo que sitúa al proyecto en la franja de complejidad alta del criterio de evaluación.

2. Entidades del modelo

El dominio se modela con cinco clases persistentes en models/, además del usuario:

Entidad	Descripción
Prompt	Texto principal con título, contenido, categoría y etiquetas libres.
Categoría	Clasificación temática para agrupar prompts.
Colección	Agrupación personal de prompts creada por un usuario.
Comentario	Mensaje libre de un usuario asociado a un prompt.
Evaluación	Puntuación de 1 a 5 con texto opcional asociada a un prompt.
Usuario	Cuenta autenticable. No cuenta como entidad de dominio.

Diagrama entidad-relación



Relaciones principales

- Un Usuario puede tener N Prompt, N Colección, N Comentario y N Evaluación.
- Un Prompt pertenece a 0..1 Categoría y tiene N Comentario y N Evaluación.
- Una Colección referencia M Prompt mediante una lista de OIDs (N a N).
- Las relaciones se modelan **por OID** (referencias indirectas), nunca por inclusión directa, para que el borrado y la carga sean transaccionalmente seguros.

3. Arquitectura y organización del código

La aplicación está organizada de forma **modular**, siguiendo el principio de un módulo por entidad tanto en la capa de modelo como en la capa de rutas:

src/

- |— app.py # Fábrica Flask, login_manager, errores, dashboard
- |— sirope_config.py # Conexión a Sirope/Redis y utilidades de OID
- |— main.py # Punto de entrada alternativo
- |— requirements.txt
- |— models/
 - | |— usuario.py
 - | |— prompt.py
 - | |— categoria.py
 - | |— coleccion.py
 - | |— comentario.py
 - | |— evaluacion.py
- |— routes/
 - | |— auth.py
 - | |— prompts.py
 - | |— categorias.py
 - | |— colecciones.py
 - | |— comentarios.py
 - | |— evaluaciones.py
- |— templates/
 - | |— base.html
 - | |— dashboard.html

```
| |— auth/
| |— prompts/
| |— colecciones/
| |— categorias/
| |— evaluaciones/
| |— errores/      # 404.html y 500.html
|— static/css/custom.css
```

Cada entidad expone:

- Una **clase de modelo** con su constructor y propiedad `oid`.
- **Funciones auxiliares** de carga, búsqueda y borrado dentro del mismo módulo.
- Un **blueprint** de rutas con CRUD y vistas asociadas.
- Sus **plantillas** Jinja2 en `templates/<entidad>/`.

La fábrica `crear_app()` (`app.py`) configura Flask, registra los blueprints, inicializa Flask-Login, define la ruta raíz, el dashboard, los manejadores de error 404 y 500 y un filtro Jinja para formatear fechas.

4. Persistencia con Sirope sobre Redis

`sirope_config.py` centraliza la creación de la instancia de Sirope, parametrizable mediante variables de entorno (`REDIS_HOST`, `REDIS_PORT`, `REDIS_DB`). También expone dos utilidades:

- `safe_oid(srp, obj)`: serializa el OID de un objeto para usarlo en URLs.
- `oid_from_safe(srp, cadena)`: reconstruye el OID a partir de su forma segura.

El uso del *safe OID* evita exponer identificadores internos de Sirope en la URL y permite construir endpoints estables del tipo `/prompts/<safe>/editar`.

5. Integridad referencial y borrado en cascada

Una de las decisiones de diseño más importantes ha sido garantizar la **integridad estructural** del almacenamiento. Cuando una entidad se borra, la aplicación se ocupa de los objetos relacionados:

- Borrar un **Prompt** elimina en cascada sus Evaluacion y Comentario asociados y lo retira de cualquier Coleccion que lo contenga (borrar_prompt_en_cascada en models/prompt.py).
- Borrar un **Usuario** (si se permitiera por interfaz) o un objeto cuyo autor desaparece se gestiona con **carga defensiva**: las plantillas y rutas usan try/except al resolver usuario_oid y muestran "Autor desconocido" si la referencia ha dejado de existir.
- Borrar una **Categoria** desasocia los prompts (su categoria_oid queda en None) sin destruirlos.
- Borrar una **Coleccion** no afecta a los prompts que contiene; solo desaparece el agrupamiento.

Toda carga de objetos relacionados está protegida con try/except, de modo que **un OID huérfano nunca rompe la aplicación**: simplemente desaparece del listado o se muestra un valor por defecto.

6. Interfaz de usuario

Navegación

- Barra superior fija con accesos a Dashboard, Explorar, Mis prompts, Colecciones, Categorías, Login/Logout.
- Tras cualquier acción (crear, editar, borrar) se redirige a una vista coherente (detalle o listado) y se muestra un mensaje flash con el resultado.
- No es necesario el botón "atrás" del navegador en ningún punto del flujo.

Frontend

- **Pico CSS** (v2, cargado desde CDN) como base de estilos: tipografía, botones, formularios y layout responsive sin configuración adicional.
- Estilos propios en static/css/custom.css para personalizar colores, navbar, tarjetas de prompt, etiquetas y mensajes flash.
- Plantilla base base.html con bloques contenido y título que el resto de plantillas extiende, garantizando coherencia visual.

Mensajes y errores

- Validaciones de formulario muestran flash de tipo error y re-renderizan el formulario con los datos introducidos para que el usuario pueda corregirlos sin reescribir.
- Plantillas dedicadas para errores 404 y 500.

7. Autenticación y permisos

- Flask-Login gestiona la sesión.
- Usuario implementa los métodos de UserMixin necesarios (is_authenticated, get_id, etc.).
- Las contraseñas se almacenan **hasheadas** con werkzeug.security (generate_password_hash y check_password_hash).
- Las operaciones de escritura (@login_required) requieren sesión iniciada.
- Solo el autor puede editar o borrar sus propios prompts, colecciones y comentarios.
- El autor de un prompt puede borrar comentarios ajenos a su prompt (moderación local).

8. Robustez

- Todas las operaciones contra Sirope están protegidas con try/except, devolviendo listas vacías o None en lugar de propagar excepciones a la vista.
- Las rutas validan los OID seguros y devuelven 404 cuando no resuelven a un objeto válido.
- Los formularios validan campos obligatorios antes de persistir.
- Los manejadores `@app.errorhandler(404)` y `@app.errorhandler(500)` evitan que el usuario vea trazas o páginas vacías.
- No se reutilizan instancias de Sirope entre peticiones: cada request abre una conexión, lo que evita estados compartidos inseguros entre módulos.

9. Ejecución

Requisitos previos

- Python 3.10+
- Redis instalado y accesible.

Instalación

```
python -m venv .venv
```

```
source .venv/bin/activate      # En Windows: .venv\Scripts\activate
```

```
pip install -r requirements.txt
```

Datos de demostración (opcional)

Para poblar la base de datos con usuarios, prompts, categorías, colecciones, comentarios y evaluaciones de ejemplo, ejecutar desde src/:

```
python seed.py
```

El script es idempotente: omite los objetos que ya existen. Tras ejecutarlo se puede iniciar sesión con ana@demo.com o luis@demo.com (contraseña demo1234).

Arranque

En una terminal:

```
redis-server
```

En otra terminal, dentro de src/:

```
flask run
```

La aplicación queda disponible en <http://127.0.0.1:5000/>.

Variables de entorno opcionales

Variable	Por defecto	Uso
SECRET_KEY	(desarrollo)	Clave secreta de Flask.
REDIS_HOST	localhost	Host del servidor Redis.
REDIS_PORT	6379	Puerto del servidor Redis.
REDIS_DB	0	Número de base de datos en Redis.

10. Manual de usuario

1. **Registro y acceso:** el usuario se da de alta en /auth/registro indicando nombre, email y contraseña. A continuación entra desde /auth/login.
2. **Dashboard:** muestra el número de prompts y colecciones propios, junto a los últimos prompts publicados por la comunidad.
3. **Explorar prompts:** listado completo con buscador por título y tags.
4. **Crear prompt:** desde "Mis prompts" o desde el menú; se pide título, contenido, categoría opcional y tags separadas por coma.
5. **Detalle de un prompt:** presenta el contenido, autor, categoría, etiquetas, comentarios y evaluaciones. Los usuarios autenticados pueden añadir comentarios y evaluaciones.
6. **Colecciones:** cada usuario puede crear colecciones temáticas y añadir o quitar prompts en ellas.
7. **Categorías:** gestión libre de categorías compartidas por todos los usuarios.

11. Decisiones de diseño relevantes

- **OID seguros en URLs:** nunca se expone el OID interno de Sirope; se usa siempre la representación segura, lo que evita errores al copiar y pegar URLs y facilita compartirlas.
- **Carga defensiva:** cualquier load envuelto en try/except. Esto cubre el caso de referencias huérfanas tras un borrado parcial.
- **Borrado en cascada explícito** para Prompt y desasociación para Categoría. Se ha preferido esta combinación porque borrar una categoría no debe destruir contenido del usuario, mientras que un prompt sin sentido arrastra sus comentarios y evaluaciones.
- **Sin recargas innecesarias:** tras cualquier mutación se redirige (patrón *POST/Redirect/GET*) para evitar reenvíos accidentales.
- **Estilos propios** en lugar de un framework completo, para mantener el bundle de estáticos mínimo.

12. Posibles mejoras

- API JSON paralela para usar la aplicación desde un cliente externo.
- Roles de administración con moderación global.
- Búsqueda por contenido completo del prompt (no solo título y tags).
- Importación/exportación de colecciones en JSON.
- Tests automáticos con pytest y Redis en memoria para CI.

13. Conclusiones

El proyecto cumple los requisitos del enunciado: cinco entidades modeladas y persistidas con Sirope/Redis, autenticación con Flask-Login, interfaz web con plantillas Jinja2, organización modular del código, navegación coherente sin recurrir al botón "atrás" del navegador, mensajes de error gestionados y borrado con integridad referencial. La arquitectura por blueprints facilita añadir nuevas entidades sin afectar a las existentes.